

# 计算机网络实验报告3-4:可靠传输协议的性能测试对比

学号:2013365 姓名:颜铭

实验要求：

在给定的测试环境下，面向可靠传输协议的细节问题进行性能测试分析。使用吞吐率和时延作为主要的参考指标。根据图表分析得到不同实现上对性能改进的方法是否在本本地传输配合路由程序的方案腺癌是合理的。

## 1. 停等机制与滑动窗口机制的性能对比

对于停等机制，在每次发送一个数据分组之后都会一直等待接收端返回相应的 ACK 响应才会继续发送下一个数据分组，而滑动窗口则会将窗口内部的所有数据分组都进行发送，直到当窗口的 base达到下一个临界点。这也就是按顺序下一个发送等待接收的数据分组，GBN 并不带有缓冲区所以窗口内的所有数据分组都默认丢失，所以在加大发包次数的代价上通过程序的循环吞吐实现加大吞吐率的算法改进。

第一个控制单一变量进行的停等机制和滑动窗口机制的综合性能对比中，使用了固定的窗口为10的大小。

为了避免实验的偶然性，对每一个实验进行五次的时延和吞吐率的测试，取其平均值为最终结果，数据分组的大小为 4096 字节。本实验所有传输的文件固定为 1.jpg，其大小为1857353bytes。总共有443个包

对于每次实验进行五次重复实验，取其平均值作为最终结果，以避免偶然情况的发生，最后一行为平均值结果，以下为统计数据：

| 次数 | 吞吐率(bytes/s) |          | 时延s      |          |
|----|--------------|----------|----------|----------|
|    | 停等机制         | 滑动窗口     | 停等机制     | 滑动窗口     |
| 1  | 19809.8      | 110256.6 | 93.7593  | 16.84573 |
| 2  | 19817.5      | 105872.2 | 93.72287 | 17.54335 |
| 3  | 19828.9      | 105783.9 | 93.66899 | 17.55799 |
| 4  | 10684.8      | 108212.4 | 173.8313 | 17.16396 |
| 5  | 13753.9      | 110811.6 | 135.0419 | 16.76136 |
| 平均 | 16778.98     | 108187.3 | 118.0049 | 17.17448 |

Table 1-1

可以看到如果在没有任何延迟时间以及丢包率的情形下，滑动窗口机制的传输效率与停等机制相比得到了较大提升了。值得注意的是由于在GBN中单窗口接收可能有滞后的处理现象导致其实完整的接收到ack的周期并不能达到接近窗口大小比10倍的倍率，但是滑动窗口在每次节省停等机制的超时等待条件下都可以较为完整地确保10倍率的关系（也就是样本中10:1地丢包超时），所以可以看到整体的倍率还是接近8倍左右的。（这里面还要考虑到耗时间的就是

UDP 向下封装发送的过程了，停等机制发送时需要经历这样的阶段，接收时还要等待这样一个阶段，而滑动窗口则是将这一部分时间重叠了不少)

接下来我们可以通过控制转包延时，在丢包率不断增大的情形下两种机制的表现。同样的对每次实验进行五次重复实验取平均值，得到的性能表格如下

| 丢包率 | 吞吐量(bytes/s) |          | 时延(s)    |          |
|-----|--------------|----------|----------|----------|
|     | 停等机制         | 滑动窗口     | 停等机制     | 滑动窗口     |
| 1%  | 19698.6      | 123026.6 | 94.28858 | 15.09717 |
| 2%  | 19245.9      | 111487.8 | 96.50642 | 16.6597  |
| 5%  | 18207.8      | 97889.4  | 102.0086 | 18.974   |
| 10% | 17582.7      | 71545.9  | 105.6353 | 25.9603  |
| 15% | 16994.2      | 19789.9  | 109.2933 | 93.85358 |

Table 1-2

以上实验设置的计时器的超时重传时间均为 0ms，以控制变量。我们进一步有表格分析得到：

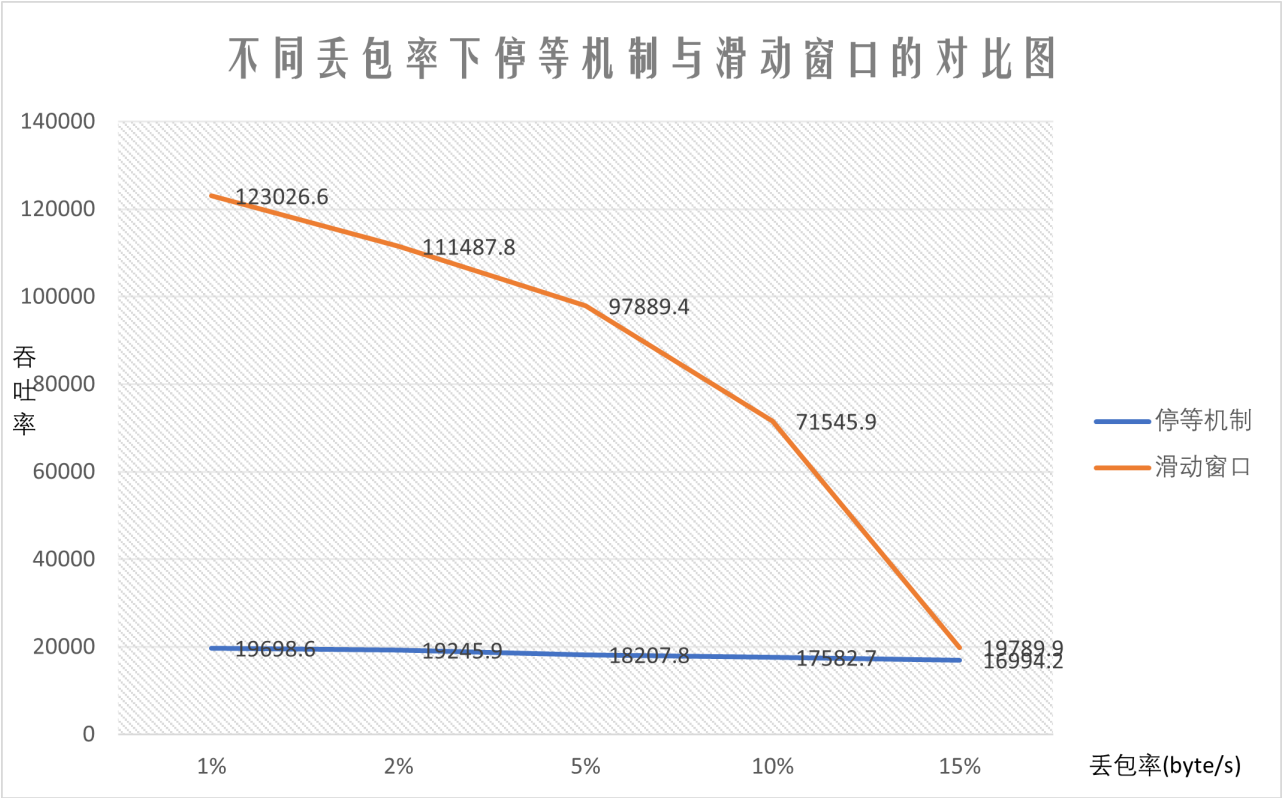


Figure 1-1

可以看到滑动窗口收到了丢包率的极大影响，当丢包率达到 15% 时基本与停等机制的效率 相同，因为滑动窗口还要重新发送窗口内的数据分组，可以设想如果丢包率进一步扩大很有可能 出现滑动窗口的效率更低的情形，因为重传的过程之中也有可能继续丢包

对于转包延迟的实验,固定丢包率为接近实际2%的丢包率后再评估吞吐率的变化，沿用有解释性的四次实验的方法我们有如下性能参数为

| 延时(ms) | 吞吐量(bytes/s) | 时延(s)    |
|--------|--------------|----------|
|        | 停等机制         | 滑动窗口     |
| 0      | 18586.5      | 111445.4 |
| 5      | 17878.6      | 45625.2  |
| 10     | 17102.3      | 56646.8  |
| 15     | 16741.1      | 34942.2  |

Table 1-3

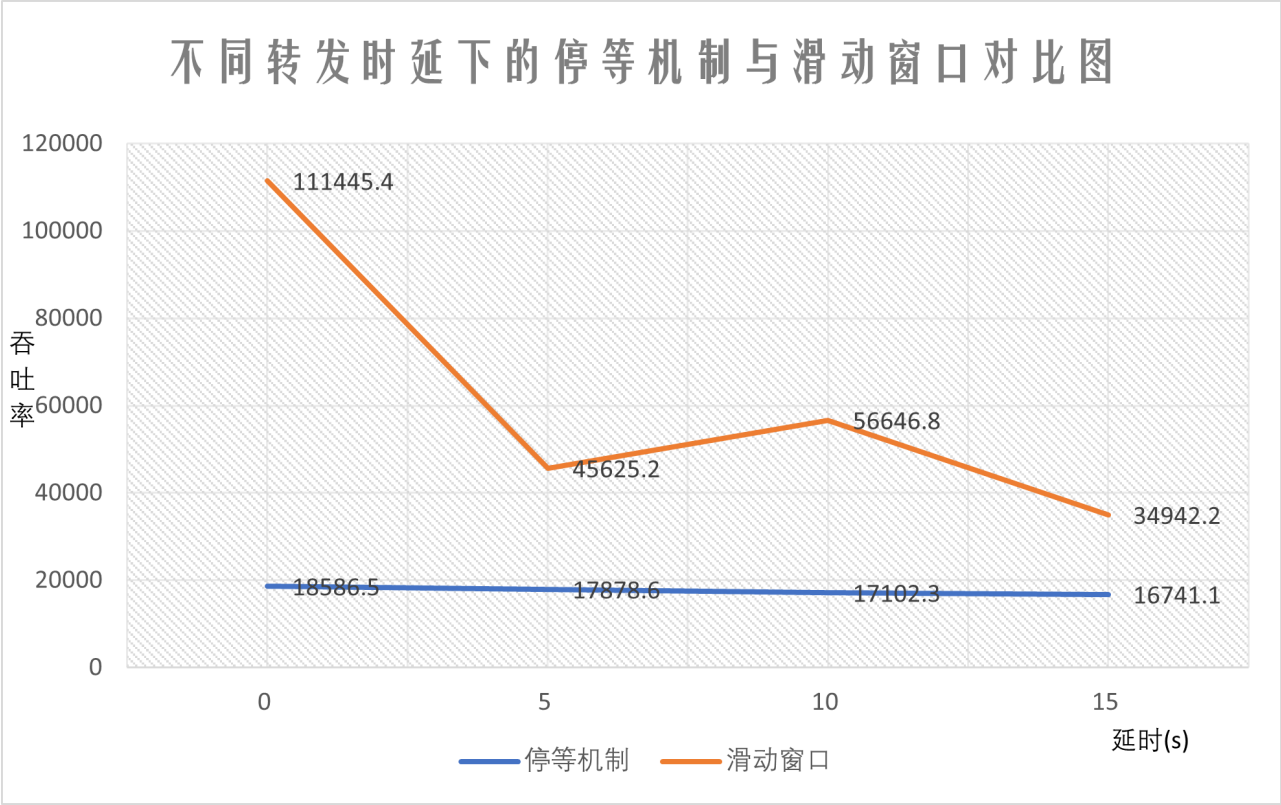


Figure 1-2

从 0ms 到 5ms 的斜率较大，是因为此时还是 UDP 向下封装传输占据了大部分的时间，而 停等机制需要一直等 ACK 响应的原因，该原因与前一部分相同。中间存在ms级别的波动，考虑到是时延可能更有瞬时吞吐意义上的触发 reno导致拥塞控制降低流量。至此，停等机制与滑动窗口机 制的分析基本完成。

2. 滑动窗口机制中不同窗口大小对性能的影响

接下来这一部分的实验分析主要是针对在滑动窗口中的不同的滑动窗口大小来进行对比分 析的。滑动窗口太小，无法充分发挥滑动窗口的优势，快速地传输文件，而滑动窗口太大又会过 多占用硬件资源 (收发两方的缓冲区过大)，并且由于接收方的缓冲区固定大小，会因为缓冲区已 满丢弃多余的分组而导致性能下降，因此要合理探讨是否 要改变窗口来调整性能

| 丢包率 | 不同窗口大小下的吞吐量(bytes/s) |         |          |          |
|-----|----------------------|---------|----------|----------|
|     | 3                    | 5       | 10       | 16       |
| 0%  | 1276126              | 1700084 | 798746   | 1472865  |
| 1%  | 52052.6              | 84020.5 | 111898.3 | 86857.1  |
| 2%  | 49291.5              | 75929.9 | 102929.9 | 108272.4 |
| 5%  | 36512.2              | 71120.8 | 93383.6  | 88382.1  |
| 10% | 19621.6              | 62220.6 | 71630.3  | 82466.8  |

Table 2-1

作图分析有:

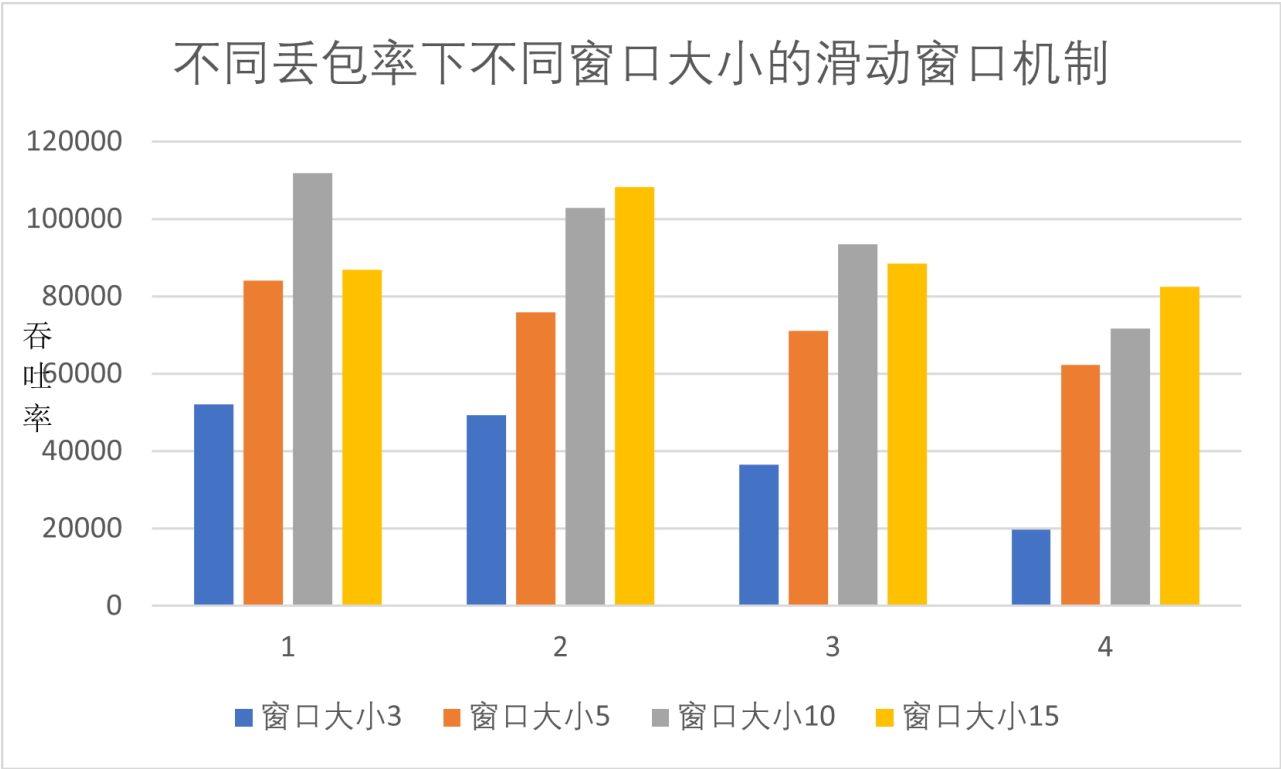


Figure 2-1

在丢包率小于等于 10% 的时候，对比窗口大小为 [3,5,10] 和 的横向数据，可以发现此时增大数据窗口能够获得一定的传输效率提升，但是我们可以发现窗口大小超过 10 之后反而没有 获得传输效率的提升，这是因为窗口大小变大之后，那么每次丢包需要重传的数据分组就会增加（对于 GBN 来说），重传的数据分组量增加就带来更多的时间消耗，并且多重传的数据分组也会 带来多的丢包情形，从而导致传输效率的下降，也就是说其实不同的丢包率下的最佳的滑动窗口 大小是不相同的。

### 3. 滑动窗口的相同窗口大小下对比累积确认和选择确认的性能

我们从滑动窗口的分析之中已经可以得知，其实滑动窗口的大小选取和实际的丢包率以及延 迟时间是息息相关的，也就是说如果能在当时的丢包率下选择合适的窗口大小就能够达到较好的 传输效率。考虑到选择确认引入了缓冲区机制不会浪费地重传反复接收的包，而这个缓冲区和包的刷新和窗口大小之间又是密切相关的，因此需要进一步验证选择确认在窗口鲁棒性的测试下是否都有明显的优劣趋势呢。

我们控制发送端的滑动窗口大小为12，进一步通过改变不同的丢包率的实验得到

| 丢包率 | 相同窗口大小 (bytes/s) |          |
|-----|------------------|----------|
|     | 累积确认             | 选择确认     |
| 0%  | 942038.5         | 1609782  |
| 1%  | 146416.4         | 170453.6 |
| 2%  | 123239.1         | 143766.9 |
| 5%  | 94237.2          | 84771.4  |
| 10% | 69089.2          | 74330.4  |

Table 3-1

分析图像为:

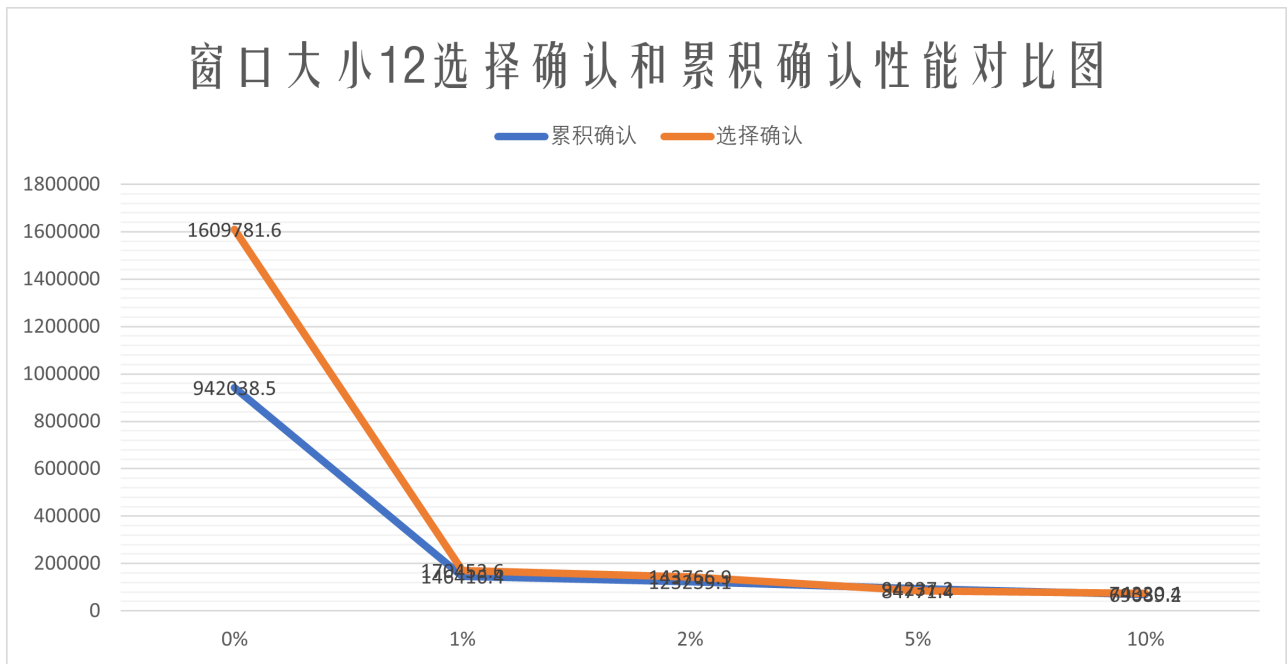


Figure 3-1

我们发现丢包率在5%前都有小幅度的提升,选择确认对累积确认不同丢包率下有提升但可能由于多线程的一些超时机制使得reno效果不见得一定比超时重传更加有效。

这是由于reno在没有出现随机丢包时,总是认为网络情况是好的,所以会不断地增加数据分组的发送数量,而当出现丢包的情形之后就会认为网络情况变差,就会减小窗口大小。并且效率的提升还取决于三次重复的ACK响应之后就会认为出现了丢包的现象,而选择确认的reno是计时器并发的,对窗口大小控制的效果可能存在不一致的偏差。

#### 4. 总结说明

本次实验之中,深入地对比分析了停等机制、滑动窗口机制和其实现上引入拥塞控制后的选择确认和累积确认的算法性能比较差异。得到了总体验证理论分析性能不断提升但是实践中存在对改进窗口大小上对上吞吐率的转发丢包下,单发送优势到验证ACK完整的过程中存在波浪式的缩放和抖动效应。

此外,对实验过程之中特殊的实验环境导致的问题进行了分析。在实现过程之中,对于多线程与单线程的相关知识进行了深入了解。参考TCP的机制,我们仍需要对基于UDP服务的可靠数据传输设计进行一定的改编与优化。